

DSA Ultimate Master Guide

Concept Explanations, Interview Intuition, and Java Solutions

1. Two Pointers

- **What it is:** Two markers scanning an array from different ends or speeds.
- **Why use it:** Reduces $O(N^2)$ time to $O(N)$ for pair searching.
- **Real-life example:** Finding matching shoes from a sorted line by checking the smallest and largest.

Problem: Two Sum II (Sorted Array)

Interview Intuition: Start at ends. If sum is small, move left. If large, move right.

Complexity: Time **$O(N)$** | Space **$O(1)$**

```
public int[] twoSum(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l < r) {
        int sum = nums[l] + nums[r];
        if (sum == target) return new int[]{l + 1, r + 1};
        else if (sum < target) l++;
        else r--;
    }
    return new int[]{};
}
```

2. Sliding Window

- **What it is:** A moving 'window' (sub-range) over an array.
- **Why use it:** Best for longest/shortest contiguous subarray problems. Avoids redundant work.
- **Real-life example:** Reading a sentence through a small magnifying glass.

Problem: Maximum Average Subarray I

Interview Intuition: Keep a running sum. Slide window by subtracting the left element and adding the right.

Complexity: Time **$O(N)$** | Space **$O(1)$**

```
public double findMaxAverage(int[] nums, int k) {
    int sum = 0;
    for(int i=0; i<k; i++) sum += nums[i];
    int maxSum = sum;
    for(int i=k; i<nums.length; i++) {
        sum += nums[i] - nums[i-k];
        maxSum = Math.max(maxSum, sum);
    }
    return (double)maxSum / k;
}
```

3. Prefix Sum

- **What it is:** Pre-calculated running total of an array.
- **Why use it:** Finds range sums in $O(1)$ time instead of $O(N)$.
- **Real-life example:** Finding expenses between Wed and Fri by subtracting Tue's total from Fri's total.

Problem: Range Sum Query

Interview Intuition: $\text{Sum}(i, j) = \text{Prefix}[j] - \text{Prefix}[i-1]$.

Complexity: Time **$O(1)$ per query** | Space **$O(N)$**

```
class NumArray {
    int[] prefix;
    public NumArray(int[] nums) {
        prefix = new int[nums.length + 1];
        for (int i=0; i<nums.length; i++)
            prefix[i+1] = prefix[i] + nums[i];
    }
    public int sumRange(int left, int right) {
        return prefix[right+1] - prefix[left];
    }
}
```

4. Fast & Slow Pointers

- **What it is:** Two pointers moving at different speeds (1 step vs 2 steps).
- **Why use it:** Finds middle element or detects cycles in Linked Lists.
- **Real-life example:** Two runners on a track. The faster one eventually laps the slower one.

Problem: Middle of the Linked List

Interview Intuition: When the fast pointer reaches the end, the slow pointer is exactly in the middle.

Complexity: Time **$O(N)$** | Space **$O(1)$**

```
public ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

5. Binary Search

- **What it is:** Continuously cutting a sorted search space in half.
- **Why use it:** Reduces search time from $O(N)$ to $O(\log N)$.
- **Real-life example:** Opening a dictionary to the middle, then ignoring the half where your word isn't.

Problem: Standard Binary Search

Interview Intuition: Check middle. If target is smaller, search left. If larger, search right.

Complexity: Time $O(\log N)$ | Space $O(1)$

```
public int search(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) l = mid + 1;
        else r = mid - 1;
    }
    return -1;
}
```

6. Backtracking

- **What it is:** Exploring a path, and undoing the choice (stepping back) if it fails.
- **Why use it:** Generates all combinations or permutations.
- **Real-life example:** Navigating a maze. Hit a dead end? Walk back to the last split and try another path.

Problem: Subsets

Interview Intuition: Recursively choose to include or exclude the current element.

Complexity: Time $O(N * 2^N)$ | Space $O(N)$

```
public void backtrack(List<List<Integer>> res, List<Integer> temp, int[] nums, int start) {
    res.add(new ArrayList<>(temp));
    for (int i = start; i < nums.length; i++) {
        temp.add(nums[i]);
        backtrack(res, temp, nums, i + 1);
        temp.remove(temp.size() - 1);
    }
}
```

7. Monotonic Stack

- **What it is:** A stack kept strictly in increasing or decreasing order.
- **Why use it:** Finds the Next Greater/Smaller element in $O(N)$ time.
- **Real-life example:** Standing in line, you can only see people taller than those in front of them.

Problem: Next Greater Element

Interview Intuition: Keep a decreasing stack. If current > stack top, it's the next greater for stack top.

Complexity: Time $O(N)$ | Space $O(N)$

```
public int[] nextGreater(int[] nums) {
    int[] res = new int[nums.length];
    Stack<Integer> stack = new Stack<>();
    for (int i = nums.length - 1; i >= 0; i--) {
        while (!stack.isEmpty() && stack.peek() <= nums[i]) stack.pop();
        res[i] = stack.isEmpty() ? -1 : stack.peek();
        stack.push(nums[i]);
    }
    return res;
}
```

8. Breadth-First Search (BFS)

- **What it is:** Exploring a tree/graph level by level using a Queue.
- **Why use it:** Finds the shortest path in unweighted graphs.
- **Real-life example:** A water drop hitting a pond, creating evenly expanding ripples.

Problem: Binary Tree Level Order Traversal

Interview Intuition: Process all nodes at the current level before queuing the next level's children.

Complexity: Time $O(V + E)$ | Space $O(V)$

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);
    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> level = new ArrayList<>();
        for (int i=0; i<size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.add(node.left);
            if (node.right != null) q.add(node.right);
        }
        res.add(level);
    }
    return res;
}
```

9. Depth-First Search (DFS)

- **What it is:** Exploring as deep as possible down one path using Recursion/Stack.
- **Why use it:** Great for counting islands, checking connectivity, and exploring all paths.
- **Real-life example:** Exploring a deep cave. Go down one tunnel until it ends, then backtrack.

Problem: Maximum Depth of Binary Tree

Interview Intuition: Find depth of left and right subtrees, take the max, and add 1.

Complexity: Time **$O(V + E)$** | Space **$O(V)$**

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    int left = maxDepth(root.left);
    int right = maxDepth(root.right);
    return Math.max(left, right) + 1;
}
```

10. Dynamic Programming (DP)

- **What it is:** Breaking problems into subproblems and storing answers (memoization).
- **Why use it:** Turns $O(2^N)$ exponential time into $O(N)$ linear time.
- **Real-life example:** Knowing $1+1+1+1+1=5$. Add '+1' and you don't recount; you just remember $5+1=6$.

Problem: Climbing Stairs

Interview Intuition: Ways to reach step i = ways to reach $(i-1)$ + ways to reach $(i-2)$.

Complexity: Time **$O(N)$** | Space **$O(1)$ optimized**

```
public int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int sum = a + b;
        a = b;
        b = sum;
    }
    return b;
}
```